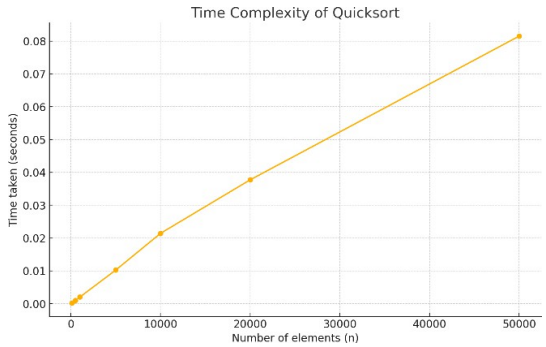
INDEX

S NO	TITLE	DATE	SUBMISSION DATE	SIGN
1	Sort elements using **Quick Sort** and analyze time complexity			
2	Implement a **Parallelized Merge Sort** and analyze time complexity			
3	(A) Obtain **Topological Order** of vertices in a directed graph (B) Compute **Transitive Closure** using **Warshall's Algorithm**			
4	Solve **0/1 Knapsack Problem** using Dynamic Programming			
5	Find **Shortest Paths** using **Dijkstra's Algorithm**			
6	Find Minimum Cost Spanning Tree using Kruskal's Algorithm			
7	(A) Print all Reachable Nodes using **Breadth-First Search (BFS)** (B) Check Graph Connectivity using **Depth-First Search (DFS)**			
8	Find Minimum Cost Spanning Tree using Prim's Algorithm			

Q1. Sort a given set of elements using the Quicksort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
import random
import time
import matplotlib.pyplot as plt

def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)

def measure_time(n):
    arr = [random.randint(0, 10000) for _ in range(n)]
    start_time = time.time()
    quicksort(arr)
    end_time = time.time()
    return end_time - start_time

# Measure and plot the time taken for different sizes of n
sizes = [100, 500, 1000, 5000, 10000, 20000, 50000]
times = []

for size in sizes:
    times.append(measure_time(size))

plt.plot(sizes, times, marker='o')
plt.title("Time Complexity of Quicksort")
plt.xlabel("Number of elements (n)")
plt.ylabel("Time taken (seconds)")
plt.grid(True)
plt.show()
```

Output:



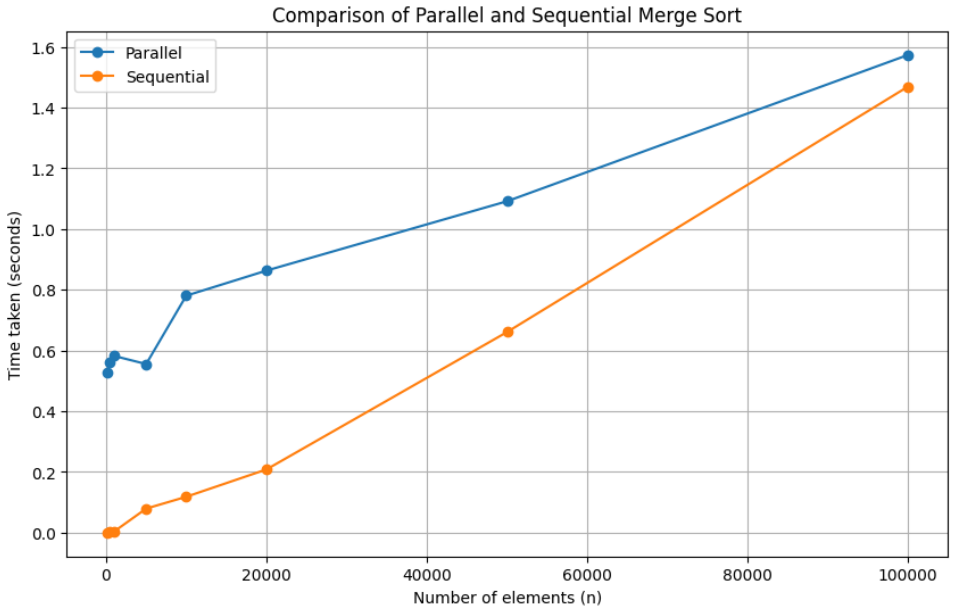
Q2. Implement a parallelized Merge Sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
import random
import time
import concurrent.futures
import matplotlib.pyplot as plt

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def parallel_merge_sort(arr, depth=0, max_depth=4):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2


    if depth < max_depth:
        with concurrent.futures.ProcessPoolExecutor() as executor:
            left_future = executor.submit(parallel_merge_sort, arr[:mid], depth+1, max_depth)
            right = parallel_merge_sort(arr[mid:], depth+1, max_depth)
            left = left_future.result()
    else:
        left = parallel_merge_sort(arr[:mid], depth+1, max_depth)
        right = parallel_merge_sort(arr[mid:], depth+1, max_depth)

    return merge(left, right)

def sequential_merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = sequential_merge_sort(arr[:mid])
    right = sequential_merge_sort(arr[mid:])
    return merge(left, right)

def measure_time(n, parallel=True):
    arr = [random.randint(0, 10000) for _ in range(n)]
    start_time = time.time()
    if parallel:
        parallel_merge_sort(arr)
    else:
        sequential_merge_sort(arr)
    end_time = time.time()
    return end_time - start_time
```

```
sizes = [100, 500, 1000, 5000, 10000, 20000, 50000, 100000]
parallel_times = []
sequential_times = []

for size in sizes:
    print(f"Testing size: {size}")
    parallel_times.append(measure_time(size, parallel=True))
    sequential_times.append(measure_time(size, parallel=False))

plt.figure(figsize=(10, 6))
plt.plot(sizes, parallel_times, marker='o', label='Parallel')
plt.plot(sizes, sequential_times, marker='o', label='Sequential')
plt.title("Comparison of Parallel and Sequential Merge Sort")
plt.xlabel("Number of elements (n)")
plt.ylabel("Time taken (seconds)")
plt.grid(True)
plt.legend()
plt.show()
```

Output :-

```
Testing size: 500
Testing size: 1000
Testing size: 5000
Testing size: 10000
Testing size: 20000
Testing size: 50000
Testing size: 100000
```



Q3. a. Obtain the Topological ordering of vertices in a given digraph.

```
from collections import defaultdict, deque

def topologicl_sort(graph):

# Step 1: Compute in-degrees of all vertices
    in_degree = {u: 0 for u in graph}
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1
# Step 2: Initialize queue with vertices having in-degree of 0
    queue = deque([u for u in in_degree if in_degree[u] == 0])

    top_order = []

    while queue:
        u = queue.popleft()
        top_order.append(u)

        for v in graph[u]:
            in_degree[v] -= 1
             if in_degree[v] == 0:
                 queue.append(v)
# Check if there was a cycle
    if len(top_order) != len(graph):
        raise ValueError("Graph has at least one cycle and thus no topological ordering is
possible.")
    return top_order

# Example usage

if __name___== "__main__":
    # Example graph represented as an adjacency list
    graph = {
        'A': ['B', 'C'],
        'B': ['D'],
        'C': ['D'],
        'D': []
    }

    top_order = topological_sort(graph)
    print("Topological Ordering:", top_order)
```

Output:

Topological Ordering: ['A', 'C', 'B', 'D']

b. Compute the transitive closure of a given directed graph using Warshall's algorithm.

def warshall_algorithm(adjacency_matrix):

```
    n = len(adjacency_matrix)  # Number of vertices
```

```
    # Initialize the transitive closure matrix as the same as the adjacency matrix
    transitive_closure = [row[:] for row in adjacency_matrix]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                # Update transitive closure
                transitive_closure[i][j] = transitive_closure[i][j] or (transitive_closure[i][k] and
transitive_closure[k][j])

    return transitive_closure

# Example usage
if __name___== "__main__":
    # Example adjacency matrix for a graph with 3 vertices
    adjacency_matrix = [
        [0, 1, 0],
        [0, 0, 1],
        [1, 0, 0]
    ]
    transitive_closure = warshall_algorithm(adjacency_matrix)

    # Print the transitive closure matrix

    print("Transitive Closure:")
    for row in transitive_closure:
        print(row)
```

Output:

```
Transitive Closure:
[1, 1, 1]
[1, 1, 1]
[1, 1, 1]
```

Q4. Implement 0/1 Knapsack problem using Dynamic Programming.

```
def knapsack(weights, values, capacity):
    n = len(weights)  # Number of items
    # Initialize the DP table with 0s
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Fill the DP table
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                # Max value of including or excluding the item
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]] + values[i - 1])
            else:
                # Item cannot be included
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

# Example usage
```

```
if _name___== "_main_":
    weights = [1, 2, 3, 8, 7, 4]
    values = [20, 5, 10, 40, 15, 25]
    capacity = 10

    max_value = knapsack(weights, values, capacity)
    print("Maximum value in Knapsack:", max_value)
```

Output:

Maximum value in Knapsack: 60

Q5. From a given vertex in a weighted connected graph, find the shortest paths to other vertices using Dijkstra's algorithm.

```
import heapq

def dijkstra(graph, start):

    # Priority queue to store (distance, vertex) pairs
    pq = [(0, start)]
    # Dictionary to store the shortest distance to each vertex
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0

    while pq:
        # Extract the vertex with the minimum distance
        current_distance, current_vertex = heapq.heappop(pq)

        # If the distance is greater than the recorded distance, skip it
        if current_distance > distances[current_vertex]:
            continue

        # Update the distance for each adjacent vertex
        for neighbor, weight in graph[current_vertex]:
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return distances

# Example usage
if __name___== "__main__":
    # Example graph represented as an adjacency list
    graph = {
        'A': [('B', 1), ('C', 4)],
        'B': [('A', 1), ('C', 2), ('D', 5)],
        'C': [('A', 4), ('B', 2), ('D', 1)],
        'D': [('B', 5), ('C', 1)]
    }

    start_vertex = 'A'
    shortest_paths = dijkstra(graph, start_vertex)

    print("Shortest paths from vertex '{}':".format(start_vertex))
```

```
    for vertex, distance in shortest_paths.items():
        print("Distance to vertex '{}': {}".format(vertex, distance))
```

Output:

```
Shortest paths from vertex 'A':
Distance to vertex 'A': 0
Distance to vertex 'B': 1
Distance to vertex 'C': 3
Distance to vertex 'D': 4
```

Q6. Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
class UnionFind:

 def __init__(self, n):  self.parent
        = list(range(n)) self.rank =
        [0] * n

    def find(self, u):
        if self.parent[u] != u:
            self.parent[u] = self.find(self.parent[u])
        return self.parent[u]

    def union(self, u, v):
        rootU = self.find(u)
        rootV = self.find(v)
        if rootU != rootV:
            if self.rank[rootU] > self.rank[rootV]:
                self.parent[rootV] = rootU
            elif self.rank[rootU] < self.rank[rootV]:
                self.parent[rootU] = rootV
            else:
                self.parent[rootV] = rootU
                self.rank[rootU] += 1

def kruskal(n, edges):
    # Sort edges by weight
    edges.sort(key=lambda x: x[2])
    uf = UnionFind(n)
    mst = []
    total_cost = 0

    for u, v, weight in edges:
        if uf.find(u) != uf.find(v):
            uf.union(u, v)
            mst.append((u, v, weight))
            total_cost += weight
    return mst, total_cost

# Example usage

if __name__== "__main__":
    # Example graph with 4 vertices and 5 edges
```

```
    n = 4
    edges = [
        (0, 1, 10),
        (0, 2, 6),
        (0, 3, 5),
        (1, 3, 15),
        (2, 3, 4)
    ]

    mst, total_cost = kruskal(n, edges)
    print("Minimum Cost Spanning Tree (Kruskal's Algorithm):")
    for u, v, weight in mst:
        print(f"Edge ({u}, {v}) with weight {weight}")
    print("Total cost:", total_cost)
```

Output:

```
Minimum Cost Spanning Tree (Kruskal's Algorithm):
Edge (2, 3) with weight 4
Edge (0, 3) with weight 5
Edge (0, 1) with weight 10
Total cost: 19
```

Q7. a. Print all the nodes reachable from a given starting node in a digraph using BFS method.

```
from collections import deque
def bfs(graph, start):

    visited = set()
    queue = deque([start])
    reachable_nodes = []

    while queue:
        vertex = queue.popleft()
        if vertex not in visited:
            visited.add(vertex)
            reachable_nodes.append(vertex)
            queue.extend(neighbor for neighbor in graph[vertex] if neighbor not in visited)

    return reachable_nodes

# Example usage
if __name__== "__main__":
    # Example digraph represented as an adjacency list
    graph = {
        'A': ['B', 'C'],
        'B': ['D'],
        'C': ['D'],
        'D': ['E'],
        'E': []
    }

    start_node = 'A'
    reachable_nodes = bfs(graph, start_node)
```

```
    print("Nodes reachable from '{}':".format(start_node))
    print(reachable_nodes)
```

Output:

```
Nodes reachable from 'A':
['A', 'B', 'C', 'D', 'E']
```

b. Check whether a given graph is connected or not using DFS method.

```
def dfs(graph, vertex, visited):
    visited.add(vertex)
    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

def is_connected(graph):
    start_vertex = next(iter(graph))
    visited = set()
    dfs(graph, start_vertex, visited)
    return len(visited) == len(graph)

# Example usage
if __name___== "__main__":
    # Example undirected graph represented as an adjacency list
    graph = {
        'A': ['B', 'C'],
        'B': ['A', 'D'],
        'C': ['A', 'D'],
        'D': ['B', 'C']
    }

    connected = is_connected(graph)
    print("The graph is connected:" if connected else "The graph is not connected.")
```

Output:

```
The graph is connected.
```

Q8. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
import heapq
def prim(graph, start):
    """
    Find the Minimum Spanning Tree (MST) using Prim's algorithm.

    :param graph: A dictionary where keys are vertices and values are lists of (neighbor,
weight) tuples.
    :param start: The starting vertex for Prim's algorithm.
    :return: List of edges in the MST and its total cost.
    """
    mst = []
    total_cost = 0
    visited = set()
    min_heap = [(0, start)]  # (weight, vertex)
```

```
    while min_heap:
        weight, u = heapq.heappop(min_heap)
        if u in visited:
            continue
        visited.add(u)
        total_cost += weight
        if weight > 0:
            mst.append((prev, u, weight))

        for neighbor, edge_weight in graph[u]:
            if neighbor not in visited:
                heapq.heappush(min_heap, (edge_weight, neighbor))
                prev = u

    return mst, total_cost

# Example usage
if __name___== "__main__":
    # Example graph with 4 vertices and 5 edges
    graph = {
        'A': [('B', 10), ('C', 4)],
        'B': [('A', 10), ('C', 2), ('D', 15)],
        'C': [('A', 4), ('B', 2), ('D', 1)],
        'D': [('B', 15), ('C', 1)]
    }

    start_vertex = 'A'
    mst, total_cost = prim(graph, start_vertex)
    print("Minimum Cost Spanning Tree (Prim's Algorithm):")
    for u, v, weight in mst:
        print(f"Edge ({u}, {v}) with weight {weight}")
    print("Total cost:", total_cost)
```

Output:

Minimum Cost Spanning Tree (Prim's Algorithm):
Edge (A, C) with weight 4
Edge (C, B) with weight 2
Edge (C, D) with weight 1
Total cost: 7